رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

Data cleaning process and organization of data with SQL

Session 2

Agenda

- **Applying Basic SQL Queries to Explore Data**

Introduction to SQL for Data Exploration

SQL as a Data Exploration Tool

SQL allows users to **extract specific information** from large datasets **efficiently**.

Key Components:

- **SELECT**: Retrieves data from a database.
- **WHERE**: Filters records based on specified conditions.
- **ORDER BY**: Organizes the result set in a specified order.

Real-World Relevance: These commands are widely used in **industries** such as **finance**, **healthcare**, and **e-commerce** to analyze **trends**, monitor **operations**, and make **informed decisions**.

Understanding the SELECT Statement

SELECT is used to specify the **columns** that you want to retrieve from a database.

Syntax:

```
SELECT column1, column2, ...
FROM table_name;
```

Example: Retrieve the **first name** and **last name** of employees.

```
SELECT FirstName, LastName
FROM Employees;
```

Filtering Data with WHERE Clause

WHERE is used to filter records based on specified conditions.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example: Retrieve employees who earn more than $50,000

```
SELECT FirstName, LastName
FROM Employees
WHERE Salary > 50000;
```

Advanced Data Filtering with WHERE

Combining Conditions: Use **AND**, **OR**, and **NOT** to create **complex** conditions.

Example: Retrieve **employees** who work in the **Sales** department and earn **more** than $50,000.

```
SELECT FirstName, LastName
FROM Employees
WHERE Department = 'Sales' AND Salary > 50000;
```

Advanced Data Filtering with WHERE

Wildcard Search: Use LIKE for pattern matching in SQL to find specific patterns within string data.

Example: The **'A%'** pattern matches any string that starts with the letter ‘A’ followed by zero or more characters.

```
SELECT FirstName, LastName
FROM Employees
WHERE FirstName LIKE 'A%';
```

Advanced Data Filtering with WHERE

IN and BETWEEN: Simplify conditions with IN and BETWEEN operators.

Example: Retrieve employees whose **salary** is **between** $40,000 and $60,000.

```
SELECT FirstName, LastName
FROM Employees
WHERE Salary BETWEEN 40000 AND 60000;
```

Sorting Data with ORDER BY

ORDER BY is used to sort the result-set in ascending or descending order.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC];
```

Example: Retrieve employee names and **sort** them by **salary** in **descending** order.

```
SELECT FirstName, LastName, Salary
FROM Employees
ORDER BY Salary DESC;
```

Sorting Data with ORDER BY

Table: Customers

customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE

```
SELECT *
FROM Customers
ORDER BY age ASC;
```

customer_id	first_name	last_name	age	country
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE
1	John	Doe	31	USA

Combining WHERE and ORDER BY for Refined Queries

Scenario: Retrieve a list of employees in the **Marketing** department, earning **more** than $**60,000**, and **sort** them by **hire date**.

```sql
SELECT FirstName, LastName, HireDate, Salary
FROM Employees
WHERE Department = 'Marketing' AND Salary > 60000
ORDER BY HireDate ASC;
```

Hands-On Exercise: Basic Data Exploration

Scenario: Retrieve a list of customers who made purchases **over** $1000 in the **last month**, **sorted** by **purchase date**.

```sql
SELECT CustomerName, PurchaseAmount, PurchaseDate
FROM Orders
WHERE PurchaseAmount > 1000
  AND PurchaseDate > '2024-07-01'
ORDER BY PurchaseDate DESC;
```

Introduction to Multi-Table Queries with JOINs

JOINs are used to **combine** rows from two or more tables based on a related column between them.

Types of JOINs: INNER JOIN, **LEFT** JOIN, **RIGHT** JOIN, and **FULL OUTER** JOIN.

Example: Retrieve employee names along with their department names.

```
SELECT Employees.FirstName, Employees.LastName, Departments.DepartmentName
FROM Employees
INNER JOIN Departments
ON Employees.DepartmentID = Departments.DepartmentID;
```

Hands-On Exercise: Multi-Table Data Exploration

Scenario: Retrieve a list of products along with the total sales for each product in the last quarter (April - June 2024)

Example Steps:

- **Identify relevant tables**: Products and Sales tables.
- **Join the tables**: Use an inner join on the product ID to match products with sales data.
- **Filter the date range**: Use a WHERE clause to select only sales from April to June 2024.
- **Calculate total sales**: Use SUM() to aggregate the total sales for each product.
- **Display results**: Select the product name and total sales.

```sql
SELECT Products.ProductName,
       SUM(OrderDetails.Quantity * OrderDetails.UnitPrice) AS TotalSales
FROM Products
INNER JOIN OrderDetails
    ON Products.ProductID = OrderDetails.ProductID
INNER JOIN Orders
    ON OrderDetails.OrderID = Orders.OrderID
WHERE Orders.OrderDate BETWEEN '2024-04-01' AND '2024-06-30'
GROUP BY Products.ProductName
ORDER BY TotalSales DESC;
```

Leveraging Subqueries for Complex Data Retrieval

Subqueries in SQL

- **Subqueries** are queries **within** a query, used to **retrieve data** that will be used in the **main query**.
- **Example**: Retrieve employees who have a **salary greater than** the **average salary** of their department.

```sql
SELECT FirstName, LastName, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary)
                FROM Employees
                WHERE DepartmentID = Employees.DepartmentID);
```

Hands-On Exercise: Advanced SQL Query Challenges

Scenario: Identify the top 5 sales representatives based on total sales in the past year, including only those who exceeded their targets

```
SELECT SalesRepName, SUM(Sales.SalesAmount) AS TotalSales
FROM SalesReps
INNER JOIN Sales
    ON SalesReps.SalesRepID = Sales.SalesRepID
WHERE Sales.SalesDate > '2023-08-01'
GROUP BY SalesRepName
HAVING SUM(Sales.SalesAmount) > (SELECT Target
                                 FROM Targets
                                 WHERE SalesReps.SalesRepID =
Targets.SalesRepID)
ORDER BY TotalSales DESC
LIMIT 5;
```

Best Practices for Effective Data Exploration with SQL

Write Clear and Concise Queries: Ensure your queries are easy to read and understand by using simple syntax.
Use Aliases: Table aliases are short names used to simplify queries, making them easier to write and read.
Comment Your Code: Add comments to explain the purpose of your queries, especially for complex logic.
Test and Optimize: Always test queries on a small set of data to check results, and improve performance by using efficient techniques (e.g., using proper indexes or avoiding unnecessary computations).
Keep Learning: SQL is a powerful tool, and continuous learning and practice are essential to mastering it.

Any Questions ?

رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

Thank You